

pyRAMBO

A Bayesian-optimization toolbox for expensive simulations

Yannick Lecomte Miguel A. Mendez

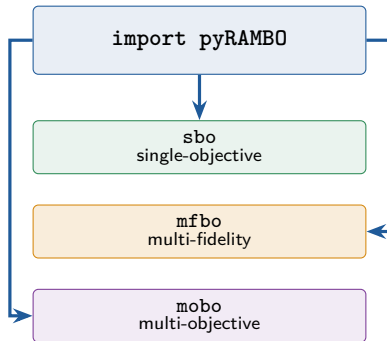
Machine Learning for Fluid Systems (ML4F)
von Karman Institute for Fluid Dynamics

October 1, 2026

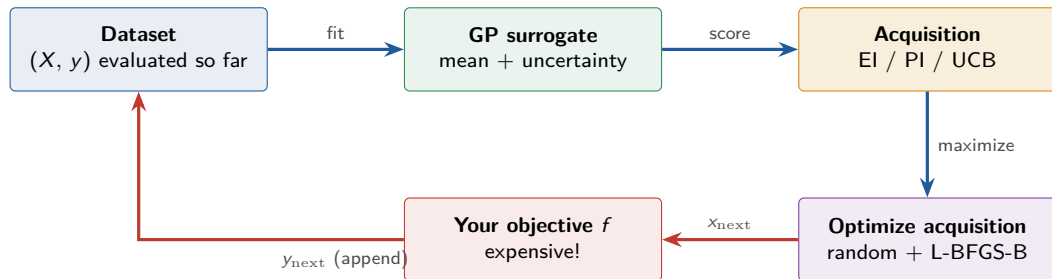
What is pyRAMBO?

Rank-one **A**ccelerated **M**ulti-objective / fidelity / output for **B**ayesian **O**ptimization.

- Gaussian-process (GP) based **Bayesian optimization** (BO) for *expensive black-box* objectives: CFD runs, experiments, training jobs.
- **Lightweight**: only numpy, scipy, matplotlib.
- **Three flavours**, one common look and feel.
- **Everything is a config object**: no hidden global state.
- Runs are **saved, logged and replayable**.



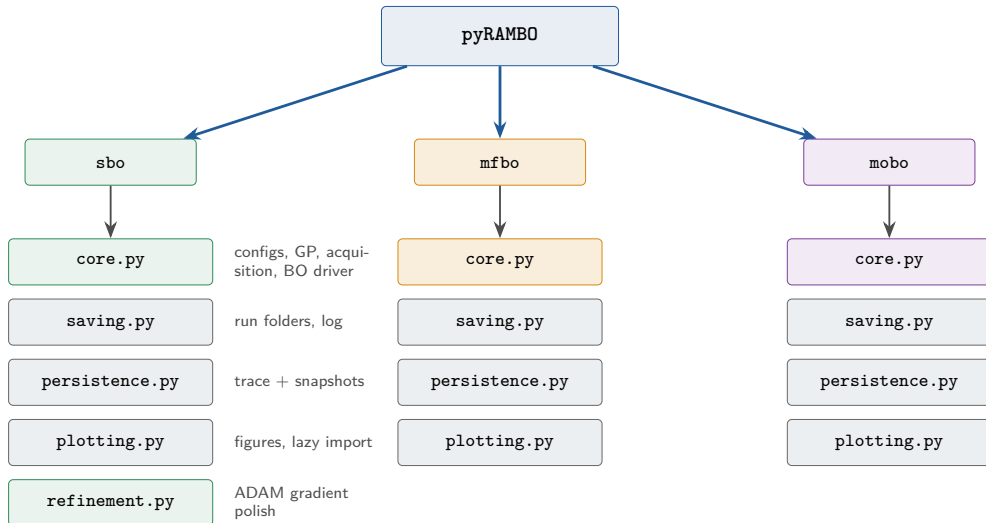
The idea in one picture: the BO loop



Cheap steps (left to right) are repeated until the evaluation budget is spent.

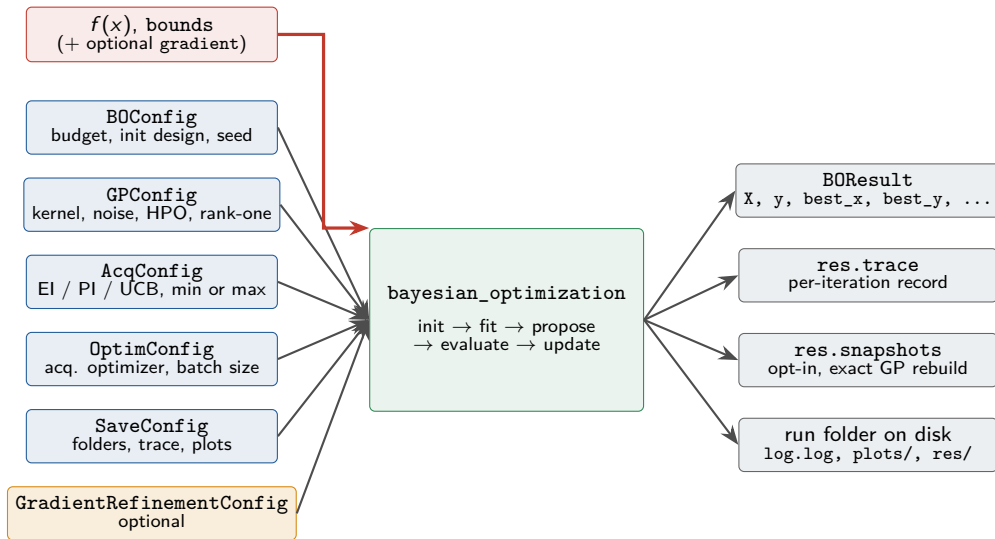
pyRAMBO implements every box of this loop, so the only thing you provide is f and the bounds.

Library architecture



Same four modules in every subpackage; the three are **independent** (no shared base, no cross imports).

Anatomy of a run: configs in, result out



Quickstart: a complete run in a few lines

```
from pyRAMBO import sbo as bo

f = lambda x: (x[0] - 0.7) ** 2    # objective to minimize

res = bo.bayesian_optimization(
    f=f,
    bounds=[(-2.0, 2.0)],
    bo_cfg=bo.B0Config(n_init=5, n_iter=15, random_state=0),
    gp_cfg=bo.GPConfig(),
    acq_cfg=bo.AcqConfig(),
    optim_cfg=bo.OptimConfig(),
    save_cfg=bo.SaveConfig(),
)
print(res.best_x, res.best_y)
```

Same pattern for the other flavours

```
from pyRAMBO import mfbo
mfbo.multi_fidelity_bayesian_optimization

from pyRAMBO import mobo
mobo.multi_objective_bayesian_optimization
```

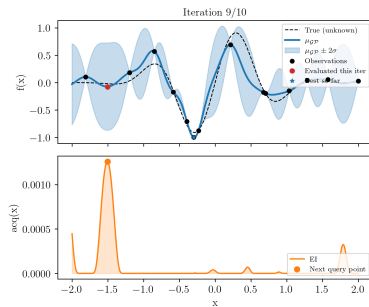
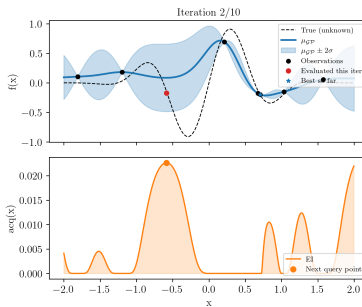
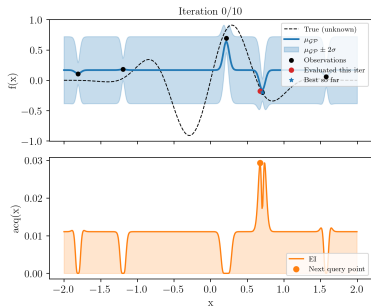
Install

```
pip install -e ".[test]"
Python ≥ 3.9.
```

Features at a glance

Feature	What you get
Acquisition functions	Expected Improvement, Probability of Improvement, Upper Confidence Bound; min or max via one flag
Hyperparameter learning	marginal-likelihood maximization (L-BFGS-B), every k iterations, with safe default bounds
Rank-one Cholesky update	extend the GP factor with new points instead of refactorizing
Initial design	Latin hypercube or uniform; or bring your own X_{init} , y_{init}
Acquisition optimizers	random, grid, or global search + local L-BFGS-B refinement
Diverse batch proposals	several well-separated candidates per iteration
Gradient refinement	polish each proposal with projected ADAM if you have adjoint/AD gradients
Multi-fidelity	cheap and expensive model combined, cost-aware (<code>mfbo</code>)
Multi-objective	Pareto front via expected hypervolume improvement (<code>mobo</code>)
Persistence & replay	per-iteration trace; opt-in snapshots to rebuild any past posterior
Built-in plots	state, convergence, history; GIF-ready frames

Seeing the loop work: 1D noisy objective (sbo)



Top: GP mean, $\pm 2\sigma$ band, observations, best so far.

Bottom: acquisition and the chosen next point.

The uncertainty collapses where we sample; EI trades off exploiting the best region against exploring wide-band regions.

Produced by the library itself:
SaveConfig(plt_all=True).

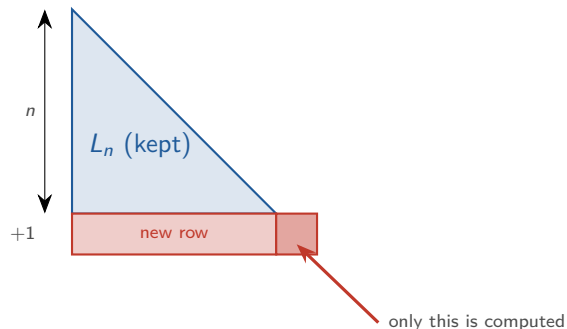
Feature: rank-one Cholesky update

Problem. Every iteration adds a point; the GP needs the Cholesky factor of the kernel matrix. Refactorizing from scratch costs $\mathcal{O}(n^3)$.

pyRAMBO. Keep the factor and *append* the new rows: $\mathcal{O}(n^2)$ per new point. Works for several new points at once (batch, refinement).

When is it used?

`GPConfig(rank_one=True, rank_one_threshold=1000)`
Only in iterations *without* a hyperparameter refit, and once the data set exceeds the threshold. A test checks it equals a full refit.



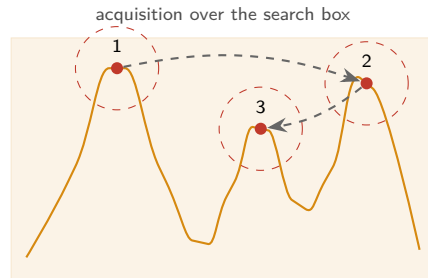
$\mathcal{O}(n^3) \rightarrow \mathcal{O}(n^2)$ per added point

Feature: diverse batch proposals

Ask for several candidates per iteration, e.g. to launch parallel simulations:

```
optim_cfg=bo.OptimConfig(  
    n_candidates=4,  
    batch_distance_scale=0.05,  
)
```

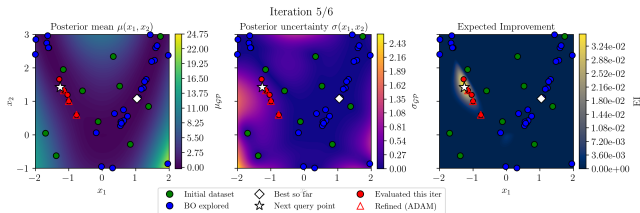
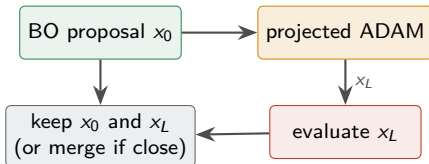
- Candidates are picked **greedily**.
- Each pick **penalizes the acquisition near itself**, so the batch spreads out.
- `n_candidates=1` is the classic loop.



Feature: gradient refinement (adjoint / AD gradients)

If your solver gives ∇f , let it finish the job:

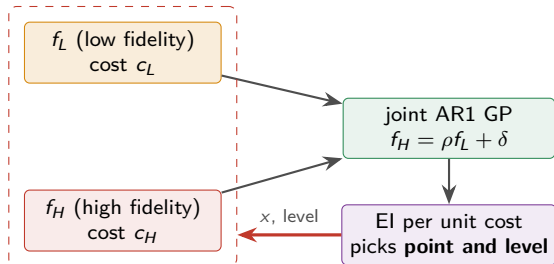
```
res = bo.bayesian_optimization(  
    f=f, gradient=grad_f, ...,  
    refinement_cfg=  
        bo.GradientRefinementConfig(  
            enabled=True,  
            n_steps=50,  
            learning_rate=1e-2),  
)
```



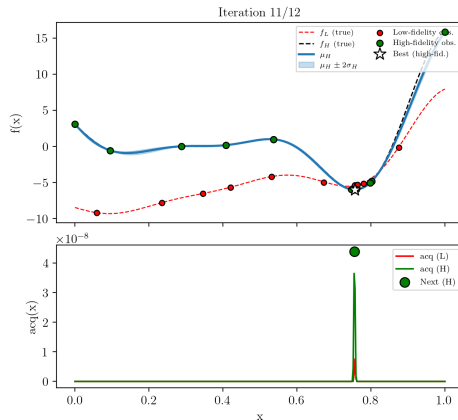
2D state plot (Rosenbrock): posterior mean, uncertainty and EI. Red triangles: ADAM-refined endpoints; red dots: points evaluated this iteration. Global search by the GP, local polish by the gradient.

mfbo: multi-fidelity optimization

Combine a **cheap, biased** model with an **expensive, accurate** one.



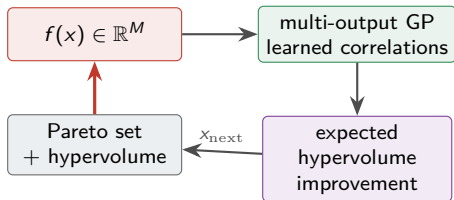
`FidelityConfig(cost_low=1, cost_high=10).`
Two fidelity levels (Kennedy & O'Hagan).



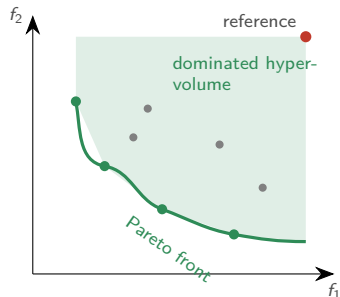
Forrester benchmark: red = cheap model and samples, green = expensive ones, μ_H = posterior of the expensive model.

mobo: multi-objective optimization

Several competing objectives $\Rightarrow$ a **Pareto front** instead of one optimum.



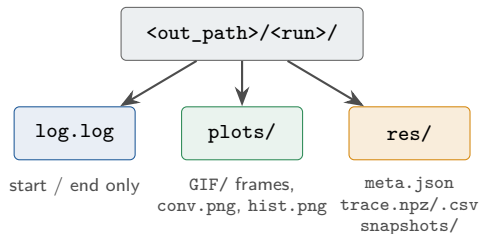
- Per-objective min/max flags (`AcqConfig(maximize=[...])`).
- Reference point built from the initial data, then kept fixed so hypervolumes are comparable over iterations.
- Anti-correlated objectives can be learned automatically.



Sketch: green = non-dominated points, grey = dominated.
BO maximizes the shaded hypervolume.

Feature: persistence and replay

Every run lands in its own folder
(`create_timestamp` or `run_<n>`); a run never overwrites another.



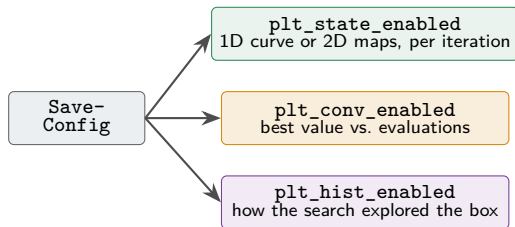
Two tiers

- **Tier 1 – trace** (always): `x`, `y`, `best`, `time`, `acquisition value`, GP hyperparameters per iteration.
- **Tier 2 – snapshots** (opt-in): what is needed to refit the GP *exactly*; never dense covariance factors.

```
cfg = bo.SaveConfig(  
    out_path="runs",  
    snapshot_enabled=True)  
res = bo.bayesian_optimization(  
    ..., save_cfg=cfg)  
run = bo.load_run(res.out_path)  
run.posterior(7).predict(Xgrid)
```

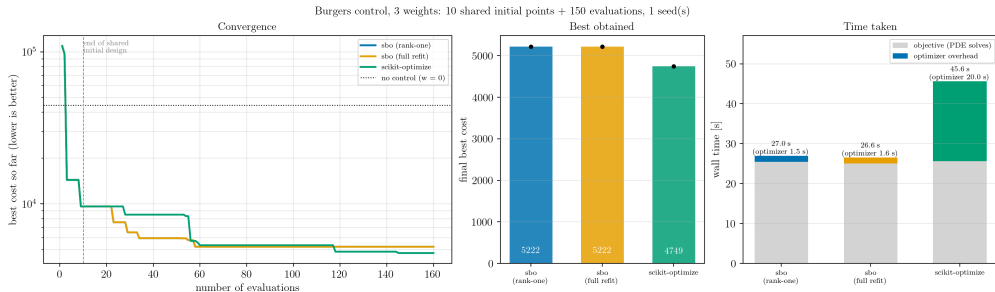
Reload months later, re-plot any iteration, no objective call. Plain numpy/JSON/CSV: no HDF5, no pickle.

Feature: built-in visualization and logging

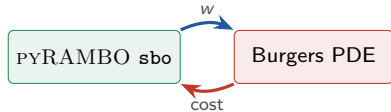


- `plt_all=True` is the master switch; `plot_every=k` thins the frames; `dpi` is configurable.
- Frames are numbered `it_XXX.png`, ready to assemble into a **GIF** of the optimization.
- The plotting module is imported **lazily**: headless / HPC runs do not need a matplotlib backend.
- The state plot marks what was evaluated this iteration and, with refinement, the ADAM endpoints.

Case study: feedback control of a Burgers wave (advanced_cases/)



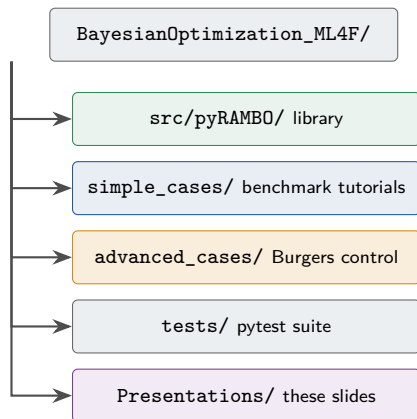
Task. Tune 3 weights w of a linear feedback law that damps a perturbed Burgers wave. One evaluation = one PDE simulation (~ 0.2 s).



Same cost and same initial design for rank-one, full refit and scikit-optimize (1 seed, 150 evaluations):

- All three beat **no control** by a large factor.
- Optimizer overhead ≈ 1.5 s for sbo vs. 20 s for skopt.
- skopt reached a somewhat lower final cost here (4749 vs. 5222): one seed, not a ranking.

Quality, repository layout and where to start



Tutorials cover: 1D and 2D benchmarks, large-scale normalization, persistence demo, Forrester multi-fidelity, Schaffer and Binh–Korn multi-objective.

Tests include a numerical check that rank-one updates reproduce a full refit, plus refinement and persistence tests.

Try it

```
pip install -e ".[test]"
pytest
start from the 1D example in simple_cases/sbo/
```

Summary and outlook

Take-aways

- One loop, **three flavours**: single-objective, multi-fidelity, multi-objective.
- Everything is a **dataclass config**; the driver is one function call.
- Speed features for expensive settings: rank-one updates, batches, gradient refinement.
- Runs are **reproducible**: folders, traces, snapshots, replay.
- Light dependencies, tested, with tutorials.

In progress (from the repo notes)

- Port the new run layout, best-so-far semantics and refinement from `sbo` to `mfbo` and `mobo`.
- More than two fidelity levels (recursive formulation).
- JOSS paper submission.

https://github.com/mendezVKI/BayesianOptimization_ML4F

Thank you – questions?